المرحلة الأولى: التأسيس

Document ID: SVR-PHASE-01

Version: 1.0

Updated: 12 November 2026

إيه هي المرحلة الأولى

المرحلة الأولى هي التأسيس. كل واحد في الفريق بيبني الجزء بتاعه على fixtures و mocks. محدش في waiting على حد. كل دور ليه شغل محدد، وكل شغل ليه نتيجة واضحة، وكل نتيجة ليها طريقة تعرف بيها إنها خلصت.

المشروع بيحوّل رسالة صوتية واحدة بالعربي المصري لعميل، لتذكرة callback مفهومة. الـ dataset جاهز. اختيار الـ models محسوم. اللي فاضل إننا نبني الـ models، نركّب النظام، نقيّمه، ونعيد تدريبه مرة واحدة على الـ labels اللي اتراجعت.

الشغل بيمشي في أربع مراحل:

- **المرحلة الأولى: التأسيس.** كل دور بيبني الجزء بتاعه على fixtures و mocks. محدش في waiting على حد.
- **المرحلة الثانية: الـ models الأساسية.** الـ text classifier والـ audio model بيتدربوا وبيتقيموا. والـ winner model بيتحدد على حسب validation mean macro-F1.
- **المرحلة الثالثة: التقييم والتكامل.** الـ test set بيتقيم مرة واحدة. منطق الـ triage بيتثبت على مخرجات حقيقية. والـ pipeline الكامل بيتكامل والـ demo بيتبني.
- **المرحلة الرابعة: إعادة التدريب على الـ labels المراجعة.** بعد ما الـ audit يخلص، الـ labels بتتصلح والـ training notebooks بتتعاد. من غير أي تغيير في الـ architecture أو الـ hyperparameters.

الـ dataset بالأرقام

الـ dataset هو SVR Voicemail Corpus. الأرقام دي هي الأساس، وكل الشغل بيتبني عليها:

- **6,522** رسالة صوتية في المجموع.
- **5,751** صف نضيف للتدريب، بعد ما اتشال منهم الصفوف المستبعدة والمكررة.
- الـ 5,751 صف دول متقسمين كده: **4,031 train** و **1,136 validation** و **584 test**. (4,031 + 1,136 + 584 = 5,751.)
- الـ seed بتاع التقسيم هو 42.

قبل ما تبدأ: من الـ terminal لأول run

كل عضو في الفريق بيمشي على الخطوات دي بالترتيب، قبل ما يبدأ شغل المرحلة الأولى. لو أي خطوة فشلت، وقف واسأل ال training leader. متحاولش تتجاوز خطوة فاشلة.

الخطوة 1: افتح الـ terminal كـ administrator

افتح الـ terminal بصلاحيات الـ administrator على جهازك. على Windows، دوس كليك يمين على الـ terminal واختار Run as administrator. على Mac أو Linux، افتح الـ terminal العادي. كل الأوامر اللي جاية بتتنفذ من الـ terminal ده.

الخطوة 2: اتأكد إن winget موجود

winget هو مدير حزم Windows، وبييجي مع Windows 11 ومع Windows 10 الحديثة كجزء من App Installer. اتأكد إنه موجود:

```
winget --version
```

لو طلع رقم إصدار، انت جاهز. لو طلع إن الأمر مش معروف، حدّث App Installer من Microsoft Store وبعدين جرّب تاني.

الخطوة 3: نزّل Git

```
winget install --id Git.Git -e --source winget
```

اتأكد إنه installed:

```
git --version
```

لازم يطلع رقم إصدار، مثلا git version 2.52. لو الأمر مش معروف بعد installed، اقفل الـ terminal وافتحه تاني كـ administrator عشان الـ PATH يتحدث.

الخطوة 4: نزّل Python 3.11

```
winget install -e --id Python.Python.3.11 --scope machine
```

اتأكد إنه installed:

```
python3 --version
```

لازم يطلع Python 3.11.x. لو الأمر مش معروف بعد installed، اقفل الـ terminal وافتحه تاني كـ administrator عشان الـ PATH يتحدث.

الخطوة 5: نزّل VS Code

```
winget install --id Microsoft.VisualStudioCode -e
```

اتأكد إنه installed:

```
code --version
```

لازم يطلع رقم إصدار VS Code. وبعدين نزّل إضافة Python الرسمية:

```
code --install-extension ms-python.python
```

الخطوة 6: اعمل clone للـ repo

```
git clone https://github.com/hazemelerefey/smart-voicemail-router.git svr cd svr
```

الخطوة 7: روح على الـ branch بتاع دورك

الكود بتاعك عايش على الـ branch بتاع دورك. كل عضو ليه أمر محدد، شغّله زي ما هو:

- حازم: `git checkout team/hazem-training-lead`
- سيرا: `git checkout team/sira-audio-speech`
- لمى: `git checkout team/lama-text-classification`
- آية: `git checkout team/aya-ml-infrastructure-evaluation`
- ابتهال: `git checkout team/ibthal-triage-routing`
- عرفة: `git checkout team/arafa-systems-demo`

الخطوة 8: هات الـ data

الـ dataset عايش على الـ branch اللي اسمه `smart-voicemail-router` ، منفصل عن فروع الكود، لأن الـ audio كبير. هاته جنب الكود بتاعك بـ worktree عشان يبقى عندك الاتنين في نفس الوقت:

```
git worktree add ../svr-data smart-voicemail-router
```

دلوقتي عندك فولدرين. `svr/` فيه الـ branch بتاع دورك والكود بتاعك. `svr-data/` فيه الـ dataset: الـ audio في `dataset/train` و `dataset/val` و `dataset/test` ، والـ notebooks في `notebooks/` ، وجداول التدريب في `final/` . فولدر الـ data ده مرجع ليك. اقرا منه. متعدّلش الـ audio ولا الجداول اللي فيه. الكود بتاعك بيشاور على فولدر الـ data بمسار، مثلا `../svr-data/dataset` .

الخطوة 9: جهّز الـ Python environment

اعمل virtual environment وفعّله:

```
python3 -m venv .venv source .venv/bin/activate
```

نزّل الـ packages الأساسية لكل الناس:

```
pip install pandas numpy scikit-learn
```

وبعدين نزّل الـ packages الزيادة اللي دورك محتاجها. كل قسم دور تحت بيقول الأمر الدقيق لدوره. اتأكد إن التنزيل اشتغل باستيراد الـ package الأساسي لدورك، مثلا `python -c "import transformers"` للّمى وسيرا.

الخطوة 10: اتأكد من الـ data

شغّل الفحوصات دي قبل ما تبدأ الشغل. كلها لازم تعدي:

1. جداول التدريب موجودة في `svr-data/final/` : clean_training_train.csv و clean_training_val.csv و clean_training_test.csv.
2. عدد الصفوف هو 4,031 train و 1,136 validation و 584 test، والمجموع 5,751.
3. فولدرات الـ audio فيها نفس الأعداد: `dataset/train` فيه 4,031 ملف، و `dataset/val` فيه 1,136 ملف، و `dataset/test` فيه 584 ملف.
4. كل ملف صوتي هو 16 kHz mono WAV وبيشتغل.

لو أي عدد غلط، وقف وقول لل training leader. متدربش على dataset انت متأكدتش منه.

الخطوة 11: اتأكد إنك جاهز تشتغل

قبل ما تبدأ شغل المرحلة الأولى، اتأكد من القايمة دي. كلها لازم تبقى صح:

1. `git --version` بيطلع رقم إصدار.
2. `python3 --version` بيطلع Python 3.11.x.
3. `code --version` بيطلع رقم إصدار VS Code.
4. فولدر `svr/` موجود وفيه الـ branch بتاع دورك.
5. فولدر `svr-data/` موجود وفيه الـ dataset.
6. الـ virtual environment متفعل، والـ packages الأساسية متنصبة.
7. أعداد الـ data صح: 4,031 train و 1,136 validation و 584 test.

لو كل ده صح، انت جاهز تشتغل على المشروع.

الخطوة 12: ابدأ المرحلة الأولى

اقرا قسم دورك اللي تحت. اعمل شغل المرحلة الأولى الأول. ادفع commits صغيرة وواضحة على الـ branch بتاع دورك وانت ماشي. حازم بيراجع ويدمج في `develop` .

الدور 1: حازم، Training Lead

Branch: team/hazem-training-lead

الأمر: `git checkout team/hazem-training-lead`

الدور ده يعني إيه

انت صاحب مجهود التدريب كله والـ model النهائي. انت الشخص اللي بيأكد إن الـ dataset والـ models والتقييم كلهم ماشيين مع بعض، وإن النتيجة النهائية حقيقية. انت بتراجع شغل كل عضو وبتدمجه. مفيش حاجة بتوصل لـ `develop` أو `master` من غير مراجعتك.

شغلك في المرحلة الأولى

جهّز الـ training pipeline. اتأكد إن جداول التدريب النضيفة بتتحمل صح وإن الـ splits مطابقة لفولدرات الـ audio. اكتب الـ training notebook skeleton اللي كل دور هيستخدمه: تحميل الـ data، والـ configuration المشتركة، والتعامل مع الـ seeds، وصيغة المخرجات للنتايج. راجع شغل المرحلة الأولى لكل عضو أول ما يوصل.

اللي غيرك محتاجه منك

- الـ training pipeline skeleton والـ configuration المشتركة، بدري في المرحلة الأولى، عشان محدش يبني على setup مختلف.
- الـ dataset، متأكد منه ومستقر، عشان محدش يدرب على data غلط.
- المراجعات والدمج، بسرعة، عشان شغل محدش يفضل متعلق من غير دمج.

تعرف إزاي إنك خلصت

الـ training notebooks بتشتغل من أولها لآخرها من غير أخطاء. والـ skeleton والـ configuration المشتركة موجودين وكل دور بيستخدمهم.

الدور 2: سيرا، Audio and Speech Engineer

Branch: team/sira-audio-speech

الأمر: `git checkout team/sira-audio-speech`

Extra packages: torch, transformers, librosa, soundfile

الأمر: `pip install torch transformers librosa soundfile`

الدور ده يعني إيه

انت صاحبة كل حاجة بتيجي من الصوت نفسه: الـ audio والـ transcription والـ audio model. شغلك إنك تثبتي الصوت بيقول إيه زيادة عن الـ transcript، وتبني الـ model اللي بيستخرج ده.

شغلك في المرحلة الأولى

اتأكدي من الـ audio. اتأكدي إن ملفات التدريب الـ 5,751 كلها 16 kHz mono WAV ومطابقة لجداول التدريب. ابني الـ intake check اللي بيرفض الـ audio البايظ وبيسجل ليه اترفض. شغّلي Whisper large-v3 على عينة وسجلي جودة الـ transcription والـ configuration المستخدمة. ابني الـ XLS-R baseline: حمّلي facebook/wav2vec2-xls-r-300m، جمّدي الـ encoder، درّبي classifier head بسيط على الـ training split، وسجلي الـ validation metrics. الـ baseline ده بيقول للفريق الصوت لوحده يقدر يعمل إيه.

اللي غيرك محتاجه منك

- الـ audio pipeline والـ intake checks، عشان الـ audio البايظ ميوصلش للتدريب من غير ما حد ياخد باله.
- الـ STT validation، عشان الفريق يعرف إن الـ transcripts موثوق فيها.
- الـ audio model والـ validation metrics بتاعته، عشان آية تقيّمه ولمى تقرر في الـ fusion.

تعرفي إزاي إنك خلصتي

الـ audio pipeline بيأكد على كل ملفات التدريب الـ 5,751. الـ STT validation متسجلة. الـ XLS-R baseline ليه validation metrics.

الدور 3: لمى، Text Classification Engineer

Branch: team/lama-text-classification

الأمر: `git checkout team/lama-text-classification`

Extra packages: torch, transformers, datasets

الأمر: `pip install torch transformers datasets`

الدور ده يعني إيه

انت صاحبة الـ model الأساسي: الـ text classifier اللي بيقرا الـ transcript وبيتنبأ بالأربع حقول. ده الـ model اللي النظام كله معتمد عليه. شغلك إنك تدربيه كويس، وتثبتيه بالـ validation، وتسلمي model جاهز للتقييم والتكامل.

شغلك في المرحلة الأولى

جهّزي الـ single-task fine-tuning. حمّلي جداول التدريب النضيفة. حمّلي الـ tokenizer بتاع EgyBERT وبتاع MARBERTv2. اتأكدي إن max sequence length 128 بيغطي الـ data: أطول transcript هو 72 كلمة، حوالي 93 token، فـ 128 أكتر من كفاية. ابني الـ training script اللي بيعمل fine-tune لـ model واحد على field واحد. شغّلي smoke test قصير على جزء صغير عشان تتأكدي إن الـ pipeline بيشتغل من أوله لآخره. اعتمدي الـ training infrastructure بتاعة آية أول ما توصل: الـ experiment tracking والـ configuration format والتعامل مع الـ seeds. متشغّليش fine-tuning كامل على setup مختلف.

نقطة الـ waiting الوحيدة

الـ fine-tuning الكامل بتاعك بيشتغل على الـ infrastructure بتاعة آية: الـ experiment tracking والـ configuration format والتعامل مع الـ seeds. آية بتسلم الـ infrastructure بدري في المرحلة الأولى، قبل الـ baseline. انت مش بتقعدي في waiting: انت بتبني الـ single-task setup وتحميل الـ data والـ tokenizer وتحميل الـ model في المرحلة الأولى، وبتشغلي smoke test على جزء صغير بـ configuration بسيطة. أول ما الـ infrastructure بتاعة آية توصل، بتعتمديها وبتشغلي الـ fine-tuning الكامل. ولو الـ infrastructure بتاعة آية اتأخرت، بتقولي لل training leader على طول عشان الخطة تتظبط.

اللي غيرك محتاجه منك

- الـ models المدربة والـ validation metrics بتاعتها، عشان آية تقيّم الـ winner model وحازم يأكد القرار.
- ترشيح الـ winner model بالأرقام اللي وراه.
- الـ configuration الدقيقة للـ winner model، عشان آية تقدر تعيد تقييم الـ test وحازم يقدر يعيد تشغيله في المرحلة الرابعة.

تعرفي إزاي إنك خلصتي

الـ single-task setup جاهز، والـ tokenizer متحمل، والـ smoke test عدّى. وانت جاهزة تعتمدي الـ infrastructure بتاعة آية أول ما توصل.

الدور 4: آية، ML Infrastructure and Evaluation Engineer

Branch: team/aya-ml-infrastructure-evaluation

الأمر: `git checkout team/aya-ml-infrastructure-evaluation`

Extra packages: scikit-learn, pandas, numpy

الأمر: `pip install scikit-learn pandas numpy`

الدور ده يعني إيه

انت صاحبة الـ machine اللي بتخلي التدريب موثوق فيه: الـ infrastructure اللي كل الناس بتشغل عليها، والتقييم اللي بيثبت إذا كانت الـ models شغالة ولا لأ. انت الشخص اللي بيأكد إن النتيجة حقيقية وقابلة لإعادة الإنتاج ومتقاسة بنفس الطريقة لكل الناس. لو رقم ميتعادش إنتاجه، فهو مش بيتحسب، وانت اللي بتكتشفي ده.

شغلك في المرحلة الأولى

ابني الـ infrastructure الأول، قبل ما حد يشغل تدريب كامل. حددي الـ configuration format: الـ model والـ hyperparameters والـ seed ونسخة الـ dataset والـ split. حددي التعامل مع الـ seeds: 5 seeds (42 و 1337 و 2023 و 7 و 99)، وكل تجربة بتشغل الخمسة وبتسجل الـ mean والـ standard deviation. حددي صيغة الـ run record: رقم الـ run والفرضية والعامل الوحيد اللي اتغير والظروف الثابتة والمقاييس والـ artifacts. ابني الـ evaluation harness: بياخد model مدرب و split، وبيشغله، وبيسجل per-field macro-F1 والـ accuracy والـ confusion matrix، بصيغة مخرجات ثابتة. ابنيه على fixtures الأول، بـ mock predictions، عشان يشتغل قبل ما أي model حقيقي يبقى موجود. وبعدين شغلي الـ TF-IDF baseline: مميزات TF-IDF مع logistic regression، model واحد لكل field، على الـ training split، وقيّميه على الـ validation split. سجلي الـ validation metrics. الـ baseline ده هو المرجع اللي الـ neural models لازم تتفوق عليه.

اللي غيرك محتاجه منك

- الـ training infrastructure، بدري في المرحلة الأولى، عشان لمى وسيرا يشتغلوا على نفس الـ setup.
- الـ evaluation harness، عشان كل الناس تقيس بنفس الطريقة.
- الـ TF-IDF baseline، عشان كل model يبقى ليه مرجع يتفوق عليه.
- الـ run records، عشان كل تجربة تبقى قابلة للتتبع.

تعرفي إزاي إنك خلصتي

الـ infrastructure متوثقة وبتستخدمها لمى وسيرا. الـ TF-IDF baseline متسجل. الـ evaluation harness بيشتغل على fixtures وعلى models حقيقية. كل تجربة ليها run record.

الدور 5: ابتهال، Triage and Routing Engineer

Branch: team/ibthal-triage-routing

الأمر: `git checkout team/ibthal-triage-routing`

Extra packages: pandas, numpy

الأمر: `pip install pandas numpy`

الدور ده يعني إيه

انت صاحبة القرار: إزاي مخرجات الـ model الأربعة بتتحول لـ queue priority وتذكرة callback يقدر الـ agent يشتغل عليها. شغلك إنك تخلي قرار النظام مفهوم. الـ agent لازم يقدر يقرا التذكرة ويعرف ليه العميل ده بيتكلم الأول، والعميل محتاج إيه.

شغلك في المرحلة الأولى

اكتبي قواعد الـ triage. ابدئي من تعريفات الحقول: complaint category ليها 11 قيمة، و complaint urgency ليها 3، و customer state ليها 5، و churn risk ليها 2. حددي الأولوية: الـ urgency العالية بتيجي الأول، والـ churn المعرض للخطر بييجي الأول، والاتنين مع بعض بييجوا قبل الكل. اكتبي القواعد كجدول، كل تركيبة من الـ urgency والـ churn risk متوصلها أولوية، عشان مفيش غموض. حددي صيغة التذكرة: مرجع العميل، والأربع حقول، والأولوية، وسبب التوجيه بكلام واضح، وسطر الافتتاح المقترح، ونسخة القاعدة اللي ساهمت. ابني مولّد التذاكر على fixtures: ادخلي له مخرجات model وهمية واتأكدي إن التذكرة اللي بيطلعها مطابقة لجدول القواعد.

اللي غيرك محتاجه منك

- قواعد الـ triage مكتوبة، عشان الفريق يراجعها قبل ما تتنفذ.
- صيغة التذكرة، عشان عرفة يبني الـ demo حواليها والـ agent يقدر يقراها.
- مولّد التذاكر، عشان عرفة يركّبه في الـ pipeline.

تعرفي إزاي إنك خلصتي

قواعد الـ triage مكتوبة كجدول كامل من غير أي تركيبة ناقصة. مولّد التذاكر بيطلع تذكرة صحيحة لكل مدخل وهمي. صيغة التذكرة متوثقة.

الدور 6: عرفة، Systems Integration and Demo Engineer

Branch: team/arafa-systems-demo

الأمر: `git checkout team/arafa-systems-demo`

Extra packages: fastapi, uvicorn, gradio

الأمر: `pip install fastapi uvicorn gradio`

الدور ده يعني إيه

انت صاحب النظام النهائي: الـ pipeline اللي بيشتغل من أوله لآخره، والـ demo اللي بيوضّح إنه شغال. شغلك إنك تاخد القطع اللي كل واحد بيبنيها وتخليها تشتغل مع بعض كنظام واحد، وبعدين تثبت إنه بيشتغل نضيف وسريع كفاية.

شغلك في المرحلة الأولى

ابني الهيكل. حدد مراحل الـ pipeline: intake و STT و classification و triage و ticket. ابني كل مرحلة كـ module منفصل بمدخل ومخرج واضحين. ابني الهيكل بـ mocks الأول: مرحلة الـ STT بترجع transcript ثابت، ومرحلة الـ classification بترجع حقول ثابتة، ومرحلة الـ triage بترجع تذكرة ثابتة. اتأكد إن الـ data بتمشي من أول مرحلة لآخر مرحلة. ابني هيكل الـ demo: رفع وتشغيل وعرض تذكرة. خلّي الـ API وعقود الـ data ثابتة عشان باقي الأدوار تقدر تبني عليها.

اللي غيرك محتاجه منك

- هيكل الـ pipeline وعقود الـ API، بدري في المرحلة الأولى، عشان كل الناس تبني components متوافقة.
- الـ demo، عشان مخرجات المسابقة تبقى حقيقية.

تعرف إزاي إنك خلصت

الـ pipeline بيمشي الـ data من أول مرحلة لآخر مرحلة بالـ mocks. هيكل الـ demo جاهز. الـ API وعقود الـ data ثابتة ومتوثقة.

مين بيعمل إيه في المرحلة الأولى

الدور	شغل المرحلة الأولى
حازم	Training pipeline skeleton والـ shared config
سيرا	Audio pipeline و STT validation والـ XLS-R baseline
لمى	Single-task setup و smoke test، وتعتمد infrastructure آية
آية	Training infra و eval harness والـ TF-IDF baseline
ابتهال	Triage rules table وصيغة التذكرة والمولّد على fixtures
عرفة	Pipeline skeleton على mocks وهيكل الـ demo وعقود الـ API

نقاط الـ waiting

المراحل متصممة عشان مفيش waiting في المرحلة الأولى، والـ waiting في المراحل اللي بعدها هو الترتيب الطبيعي للشغل، مش تعطيل.

- **لمى في waiting على infrastructure آية.** الـ fine-tuning الكامل بتاع لمى بيشتغل على الـ infrastructure بتاعة آية. آية بتسلمها بدري في المرحلة الأولى. لمى مش بتقعد في waiting: هي بتبني الـ setup والـ smoke test الأول، وبعدين بتعتمد الـ infrastructure أول ما توصل. ولو اتأخرت، بتقول لل training leader على طول.
- **ابتهال في waiting على مخرجات حقيقية عشان تثبت الـ triage.** هي بتكتب القواعد وصيغة التذكرة في المرحلة الأولى على مخرجات وهمية. مفيش model محتاجاه لكده. هي بتثبت القواعد في المرحلة الثالثة على تنبؤات الـ test الحقيقية.
- **عرفة في waiting على components حقيقية عشان يركّبها.** هو بيبني الهيكل والـ demo في المرحلة الأولى بـ mocks. مفيش component محتاجه لكده. هو بيبدّل الـ mocks بالمكونات الحقيقية أول ما توصل في المرحلة الثانية والثالثة.
- **آية في waiting على models عشان تقيّمها.** هي بتبني الـ evaluation harness في المرحلة الأولى على fixtures. وهي بتشغل الـ TF-IDF baseline في المرحلة الأولى، وده مش محتاج أي neural model. هي بتقيّم الـ models الحقيقية في المرحلة الثالثة.
- **الـ fusion في waiting على audio features سيرا.** لو الـ fusion هيتعمل في المرحلة الثانية، لمى بتحتاج الـ audio features بتاعة سيرا. الـ fusion اختياري وبيتقرر بالأدلة: لو الـ audio baseline مبيضيفش حاجة حقيقية فوق الـ text لوحده، مفيش fusion ومفيش waiting. محدش بيتعطل بسبب الـ fusion.

قاعدة الـ waiting

لو انت في waiting على مخرجات عضو تاني، متقفش. ابني على fixtures أو mocks، اكتب الـ tests، وثّق الـ interface، وجهّز التكامل. ولما المخرجات الحقيقية توصل، بدّلها واختبر. ولو الـ waiting طوّل عن المتوقع، قول لل training leader. محدش في waiting في صمت.

الـ audit ماشي بالتوازي

الـ label audit بيشتغل كل يوم بالتوازي مع كل المراحل الأربعة. كل عضو بيراجع الـ labels على صفحة الـ audit الحية كل يوم. الـ audit مش بيوقف التدريب.

إزاي تعرف إن المرحلة الأولى خلصت

المرحلة الأولى خلصت لما:

1. الـ training pipeline skeleton والـ shared config موجودين، وكل دور بيستخدمهم.
2. سيرا أكدت على الـ 5,751 ملف صوتي، والـ XLS-R baseline متسجل.
3. لمى جهزت الـ single-task setup، والـ smoke test عدّى.
4. آية سلمت الـ training infrastructure والـ TF-IDF baseline متسجل.
5. ابتهال كتبت جدول قواعد الـ triage وصيغة التذكرة، والمولّد بيشتغل على fixtures.
6. عرفة بنى هيكل الـ pipeline على mocks، وهيكل الـ demo جاهز، وعقود الـ API ثابتة.
7. حازم راجع ودمج شغل المرحلة الأولى لكل عضو في `develop`.

قواعد الشغل

القواعد دي بتطبق على كل الناس، في كل المراحل.

- **الـ branches.** اشتغل على الـ branch بتاع دورك بس. متعملش commit مباشرة على `develop` أو `master`. حازم بيدمج في `develop`، وبيدمج النتيجة النهائية المتأكدة في `master`.
- **الـ notebooks أولا.** كل الشغل بيعيش في Jupyter notebooks لحد ما المشروع كله يخلص، وبعدين بس بيتحول لـ .py. الـ notebooks لازم تتنفذ وتتحفظ بمخرجاتها.
- **الـ splits.** train و validation و test. النسبة 70/20/10.
- **متغير واحد بس في كل تجربة.** كل تجربة بتغيّر عامل واحد بس، وباقي العوامل ثابتة. لو غيّرت أكتر من عامل في نفس الوقت، مش هتعرف أي واحد فيهم هو اللي عمل الفرق. الـ run record بتاع كل تجربة بيسجل العامل الوحيد اللي اتغير والظروف الثابتة.
- **مفيش ادعاء من غير قياس.** أي رقم بتقوله لازم يكون متقاس ومتسجل مع الـ configuration والـ hardware. مفيش حاجة بتتقال إنها خلصت من غير دليل.